



OPEN Block segmentation in feature space for realtime object detection in high granularity images

Ashutosh Vijay Kotwal

Computer vision has applications in object detection, image recognition and classification, and object tracking. One of the challenges of computer vision is the presence of useful information at multiple distance scales. Filtering techniques may sacrifice details at small scales in order to prioritize the analysis of large-scale features of the image. We present a strategy for coarse-graining multidimensional data while maintaining fine-grained detail for subsequent analysis. The algorithm is based on fixed-size block segmentation in the feature space. We apply this strategy to solve the long-standing challenge of detecting particle trajectories at the Large Hadron Collider in real time.

Keywords Image segmentation, Fast pattern recognition, Multiscale image analysis, Object tracking, Computer vision, Particle tracking, Large hadron collider

A challenge of computer vision is the analysis of images that contain relevant detail on multiple distance scales. A popular strategy is based on convolutional neural networks. Details at small scales, such as edges and contours, are extracted by convolving kernels over the image to build feature maps. The pooling layers and fully connected layers assemble and recognize large-scale patterns from these small-scale features. Other approaches are based on Restricted Boltzmann Machines and Autoencoders. Object detection in images with multiple objects is usually handled by preprocessing the image to define overlapping regions or windows such that the aforementioned classifiers can be run on each window. This is the important step of image segmentation.

Although unsupervised learning has been used for computer vision, modern methods (see^{1–3} for reviews) are generally based on deep neural networks that contain hundreds of millions of parameters and use supervised learning to train on previously classified data. They require large volumes of high-quality training data and significant computing resources for training and execution.

We propose an alternate strategy that is similar to the block-spin construction in statistical physics⁴ which helped develop the idea of renormalization^{5,6} that revolutionized the understanding of physical systems at different scales. We start with the typical step of extracting a feature map from the original image by convolution with an appropriate kernel. Our use of the feature map differs from other approaches. We collect small-scale features into two-dimensional sets, called patches. We cover the entire feature map by an overlapping set of patches, i.e. every feature must be an element of at least one patch.

We describe an algorithm for creating this cover. The only assumption is that a finite number of small-scale features are needed to reconstruct and recognize large-scale objects of interest. The cover enables the preservation and analysis of both the small-scale and large-scale structure in the image.

Patches must satisfy two rules. (i) Each patch must contain a fixed number of features, i.e. all patches have a fixed cardinal number. (ii) Every object in the image must be contained in at least one patch. Object containment is defined as the containment of all features associated with the object. A patch serves a similar purpose in our strategy as regions or windows in other image-segmentation algorithms. However, rule (i) is unique to our strategy.

The novelty of our proposal is four fold. First, the cover guarantees that all objects of interest are contained in at least one patch. Second, since each patch contains a fixed number of features, the subsequent processing of all patches can be deterministic and identical⁷; this aspect allows parallelization in computing⁸. Third, the reconstruction of simple objects from their constituent features can use unsupervised learning methods such as clustering, obviating the need for vast amounts of training data and trainable parameters. Importantly, our strategy can lead to an interpretable method of computer vision that avoids the black-box approach of deep learning. Fourth, our strategy is robust in the presence of noise.

Department of Physics, Duke University, Durham NC, 27708, USA. email: ashutosh.kotwal@duke.edu

Application

We present a use case where this strategy is successful. Particle physics experiments at the Large Hadron Collider (LHC) generate vast quantities of imaging data. Collisions between high-intensity beams of protons at a frequency of 40 MHz produce $\approx 20,000$ particles every 25 ns. These particles are tracked by concentric cylindrical layers of silicon sensors that surround the beam axis. The passage of the charged particles is electronically recorded as points (“hits”) by two-dimensional pixel sensors or strip sensors. In the upgraded versions of the ATLAS and CMS experiments^{9,10} intended to start operation in 2028, 4–5 barrels of inner pixel sensors and about 10 barrels of outer strip sensors are expected. With pixel dimensions of $50\ \mu\text{m} \times 50\ \mu\text{m}$, a large point cloud of about 100,000 points is recorded, which constitutes a three-dimensional image containing the discretized helical trajectories of the particles⁷. A schematic diagram of a generic pixel detector is shown in Fig. 1.

The goal of the reconstruction algorithm is to “connect the dots”, i.e. identify radially distributed clusters of hits such that each cluster marks a particle’s trajectory through the sensor layers. With prior knowledge of the pixel positions to micrometer accuracy, each clustered set of hits can be fitted to infer the helix parameters and thence the particles’ positions and momentum vectors. In the context of experimental particle physics, the 3D point cloud is a digital image (more accurately, the feature map) and the particle trajectories, called “tracks”, are the objects of interest to be detected.

The multiplicity of particles and hits is largest in the pixel detectors, hence image reconstruction is more challenging here than with the strip sensors at larger radius. As the techniques we discuss here are equally applicable to the latter, we describe image reconstruction using the pixel-sensor hits. The physics advantages of focusing on the pixel sensors are discussed in^{7,11}.

Comparison with other methods

In the context of particle tracking at the LHC, other methods based on associative memory, the Hough transform, neural networks, or finding tracklets in paired sensors have been compared in detail with our method in^{7,11}. An advantage of our method is that it is designed to operate at the 40 MHz collision rate in real time. The ability to find all tracks in a patch with a latency of 250 ns and throughput of 40 MHz at the LHC has already been demonstrated in⁸. Based on that experience, the patch-making algorithm presented here can similarly be parallelized and executed using a small set of instructions - arithmetic operations and comparisons only. A preliminary investigation shows that the patch-making algorithm can match the latency and the 40 MHz throughput of the track-finder; this will be shown in a future publication.

The techniques based on Kalman filter, associative memory, Hough transform and neural networks are being pursued for an input rate of at most 1 MHz¹², much slower than our target rate. Particle tracking based on the Kalman filter approach is currently deployed at the LHC to operate at 100 kHz. The tracklet approach can process input data at 40 MHz but requires a special sensor configuration with pairs of closely spaced strip layers¹³; our approach is compatible with any cylindrical sensor configuration and can be deployed with pixel or strip sensors.

It should be noted that Kalman filters and similar inclusive tracking approaches do not impose a minimum p_T threshold on tracks. In the interest of speed and generality, our method imposes a modest minimum- p_T threshold to simplify the arithmetic operations, as described in detail in⁷. However, these arithmetic simplifications are not essential to our method; they may be eliminated if a cost-benefit analysis provides the motivation to reconstruct the lowest p_T tracks. The most promising applications in particle physics at the LHC motivate a minimum p_T

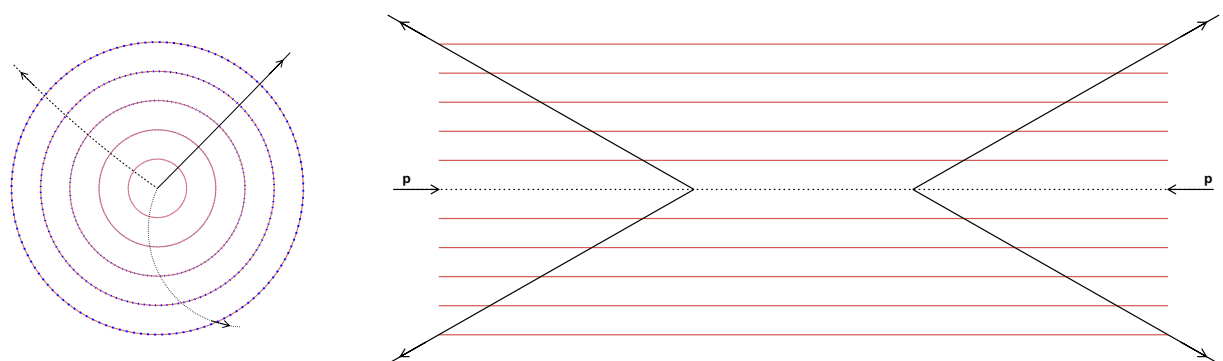


Fig. 1. Schematic of a generic pixel detector at a colliding-beam experiment. Five cylindrical layers of silicon pixel sensors (shown in red), each 1 m long and radially spaced 5 cm apart, are concentric with the longitudinal (beam) axis. Not shown is the large concentric solenoid that produces an axial magnetic field which bends the charged particles in the azimuthal direction. (Left) Three particles, of transverse momenta of 10 GeV/c (solid curve), 1 GeV/c (dashed curve) and 0.1 GeV/c (dotted curve) respectively, produced in a pp collision are shown traversing the sensor layers in the transverse (azimuthal) view. The azimuth is divided into 128 wedges as described in the text; the overlaps between adjacent wedges (shown in blue) are due to their arc-shaped boundaries designed for containment of $p_T > 10$ GeV/c particles. (Right) In the longitudinal (z) view, four particles, each of transverse momenta of 20 GeV/c, are shown. The horizontal dotted line indicates the beam axis. Example collision points are at $z_0 = \pm 15$ cm, the ends of the luminous region, and the particles pass through the edge of the acceptance of the outermost sensor layer.

threshold⁷. For this reason, methods based on associative memory, Hough Transform, neural networks and tracklet reconstruction have been pursued, even though they also incorporate minimum- p_T thresholds.

Beyond the particle tracking application at the LHC, our method differs from other approaches for image segmentation in the broader arena of computer vision. Our approach follows the philosophy that objects can be considered as composites of simple geometric constructs such as line segments and ellipses. These simple constituents can be parameterized analytically without resorting to the black-box approach of huge, trained neural networks. In future work, we plan to study a use case in computer vision where the images are amenable to such decomposition. In such cases, our algorithm is expected to compare favorably with other fast algorithms such as YOLO (You Only Look Once)(see¹⁴ for a review), especially since the latter is not as effective in detecting small objects whose simplicity favors our algorithm.

Another important aspect of our algorithm is that it is designed to operate in a noisy environment, where most of the detected features are not associated with actual objects of interest. Noisy images containing many small and simple objects would be most suitable for our algorithm, especially if high speed is desirable. The absence of any training requirement could be a significant added benefit of our algorithm.

Methods

The hits in the pixel sensors are analogs of small-scale features in a visual image, since the hit resolution is $\mathcal{O}(10\text{ }\mu\text{m})$ and two particles can record hits separated by $\mathcal{O}(100\text{ }\mu\text{m})$. Analogously to Kadanoff’s block spins⁴, which are built out of a block of N adjacent spins in a lattice, we build a patch out of N adjacent hits in each of L pixel layers at consecutive radii. Without loss of generality, we consider $L = 5$ pixel layers that are planned for the upgraded ATLAS experiment at the LHC. For reasons discussed below, we select $N = 2^n$ for a convenient integer n such as $n = 4$. The comparison with $n = 3$ and $n = 5$ will be presented to justify this choice.

The hits in the cylindrical sensor layers of radii r_l , $l \in \{1, 2, \dots, L\}$ are recorded at azimuthal (ϕ) and longitudinal (z) coordinates. We slice this three-dimensional distribution into two-dimensional wedges; each wedge contains hits inside a ϕ -interval at all (r_l, z) values. The size of the ϕ -interval grows with r_l to ensure containment of particle trajectories curving in an axial magnetic field, i.e. any particle following a circular arc in the (r, ϕ) view intersects all sensor layers within the same wedge. The curvature of the arc-shaped wedge boundaries sets the threshold on the transverse momentum (p_T) of the particles that can be reconstructed; lower- p_T particles will traverse at least two adjacent wedges and violate the containment requirement. The number of azimuthal wedges and the p_T threshold are parameters of choice; lowering the p_T threshold causes the overlap between adjacent wedges to increase. In the solution presented for real-time particle reconstruction at the LHC, we choose 128 azimuthal wedges and the p_T threshold of 10 GeV/ c for benchmarking, and we investigate variations of these values in Table 1. This threshold is low enough to maintain full sensitivity to most physics processes of interest at the LHC⁷. For this threshold, the azimuthal overlap between adjacent wedges is $\sim 15\%$, which grows with the number of wedges and represents a duplication of computing resources. As wedges can be processed in parallel, the number of wedges is chosen as a compromise between minimizing resource wastage and maximizing parallelism.

Each wedge serves as a two-dimensional image and each high- p_T trajectory is an object of interest. A small fraction of hits are associated with these trajectories; most hits act as noise.

In the (r, z) view the high- p_T trajectories can be approximated by straight lines. This characteristic is used to assign hits in a wedge to patches. The straight-line approximation is accurate to 1% or better in the pixel (strip) sensors for $p_T \sim 1$ (4) GeV/ c and improves quadratically with increasing p_T . The implied minimum p_T threshold is low enough to have negligible impact on the physics potential of the experiments. Importantly, since the small deviation of the trajectory from a straight line is calculable analytically, a corresponding adjustment is straightforward to incorporate into the containment logic if justified by a cost-benefit analysis.

In each layer l , we define a superpoint (S_l) as a set of N adjacent hits ordered in their z coordinate. A patch is a set $P = \{S_1, S_2, \dots, S_L\}$. The cardinal number of P equals $N \times L$, automatically satisfying rule (i).

Patch construction is performed by mapping the (r, z) space of hit locations to a space of parameters that describe the objects to be contained in each patch. In our use case, the parameter space is that of straight lines, which we choose to describe in terms of z_1 and z_L , the intersection of an (r, z) line at the innermost and outermost layers, respectively. Thus, each particle trajectory corresponds to a point in the (z_1, z_L) space,

Metric	n_W			$p_T^{\text{threshold}}(\text{GeV})$			n		
	64	128	256	5	10	20	3	4	5
n_{TP}	269.5 $\pm$ 60.2	69.7 $\pm$ 22.5	16.8 $\pm$ 10.0	98.7 $\pm$ 27.8	69.7 $\pm$ 22.5	56.5 $\pm$ 19.5	219.9 $\pm$ 81.3	69.7 $\pm$ 22.5	26.4 $\pm$ 18.6
n_P	52.2 $\pm$ 7.7	16.7 $\pm$ 2.9	5.8 $\pm$ 1.3	22.2 $\pm$ 3.7	16.7 $\pm$ 2.9	14.2 $\pm$ 2.5	76.0 $\pm$ 13.7	16.7 $\pm$ 2.9	4.9 $\pm$ 0.9

Table 1. The dependence of n_P and n_{TP} on n_W , the number of azimuthal wedges, and on $p_T^{\text{threshold}}$, and on the number of features per patch $N \equiv 2^n$. The simulated data correspond to $\mu = 200pp$ collisions per beam crossing at the LHC. The variation with n_W for $n_W = 64, 128, 256$ is studied for $p_T^{\text{threshold}} = 5, 10, 20$ GeV/ c respectively, thereby maintaining the shape of the wedge as its width (and the number of hits in it) is varied. The variation with $p_T^{\text{threshold}}$ and with $N \equiv 2^n$ (for $p_T^{\text{threshold}} = 10$ GeV) is investigated for a fixed $n_W = 128$. Curvature is determined for an axial magnetic field of 2 T. The mean and RMS of the distribution of n_P and n_{TP} were evaluated for simulated data of 6400 wedges and are shown in the table in the format “mean $\pm$ RMS”. The fractional statistical uncertainty ($\sim 1\%$) on these values is negligible.

$$z_l = z_1 + \frac{z_L - z_1}{r_L - r_1}(r_l - r_1)$$

where (r_l, z_l) represents the hit position on the layer l .

The luminous region in which protons collide extends over the interval $-15 \text{ cm} < z_0 < 15 \text{ cm}$, where z_0 refers to the longitudinal coordinate on the beam axis at $r \approx 0$. Particles emerge from the luminous region with z_0 values in this interval. The acceptance of cylindrical sensors extends over the intervals $-50 \text{ cm} < z_l < 50 \text{ cm}$, $l \in \{1, 2, \dots, L\}$. Ensuring acceptance of 100% for particles implies covering the parameter space bounded by $|z_0| < 15 \text{ cm}$ and $|z_L| < 50 \text{ cm}$.

As an (r, z) point in real space maps to a line in the (z_1, z_L) parameter space, any z -interval at fixed r maps to a strip in parameter space. The parameter space to be covered by patches is bounded by a parallelogram defined by the intersection of the two strips $|z_0| < 15 \text{ cm}$ and $|z_L| < 50 \text{ cm}$, respectively. We refer to the parameter space inside this bounding parallelogram as the field.

Rule (ii) requires that every particle be contained in at least one patch. Equivalently, every point in the field should be in at least one patch. In other words, the set of patches should fully tile or cover the field. An efficient algorithm achieves this goal with a small number of patches.

Each superpoint maps to a strip in the parameter space. The S_1 strip is vertical, the S_L strip is horizontal, and the intermediate layers generate strips that rotate monotonically with l . The notation S_l has two equivalent interpretations. In the context of sensors, it denotes a set of N contiguous hits on the sensor layer l . In the context of parameter space, it denotes the corresponding strip.

The straight-line trajectory of a particle should pass through all superpoints in a patch. Therefore, the acceptance of a patch for straight lines maps to the intersection of the superpoints' strips in parameter space. The intersection of L strips is a convex (Each strip is a convex set. The intersection of convex sets is convex.) polygon. We wish to create polygonal patches that are the most convenient for tiling. By minimizing overlap, we create a cover with a small number of patches.

We devise the following algorithm to cover the field. Most patches are created in pairs, consisting of a seed patch and its complementary patch. Occasionally, the pair is supplemented by an overlapping tertiary patch. The three-part sequence of operations to make a doublet/triplet of patches is iterated until the cover is achieved.

Part I - seed patch

The first patch is created from superpoints that are right-justified to the $(z_0 = 15 \text{ cm}, z_L = 50 \text{ cm})$ line in real space. This "seed" patch has a right-angled corner in parameter space that coincides with the corner of the field at these coordinates. In the simplest case, the seed patch is a rectangle in parameter space, with its sides defined by the strips S_1 and S_L when these superpoints have the most restrictive acceptance.

It is convenient to visualize the parameter space as Cartesian with z_1 on the x axis and z_L on the y axis. The upper right corner of the first seed patch (c_b) matches the upper right corner of the field. We define c_a as the top left corner and c_c and c_d as the bottom left and bottom right corners, respectively, of a patch. For non-rectangular patches, ambiguities in corner definitions are resolved below.

If the i^{th} seed patch (P^i) is rectangular, the next patch (P^{i+1}) is predictable; the latter's superpoints are right justified to the real line c_d^i by choosing $S_1^{i+1} = S_1^i$ and building the remaining superpoints such that $c_b^{i+1} = c_d^i$ and $c_a^{i+1} = c_c^i$. In other words, if the seed patch is rectangular, the next patch is built below it in parameter space with a shared horizontal side. This step also motivates the next step of making a complementary patch if the seed patch is not rectangular.

Part II - complementary patch

A rectangular seed patch is created when S_1 and S_L are the most restrictive superpoints,

$$(S_1 \cap S_L) \subset (S_2 \cap S_3 \cap \dots S_{L-1}) \implies P \equiv (S_1 \cap S_2 \cap \dots S_{L-1} \cap S_L) = (S_1 \cap S_L)$$

, which is a rectangle in the parameter space.

When the seed patch P^i is not rectangular, this is because S_L is not the most restrictive. We choose S_1 as the anchor for the patch, i.e., the patch is equivalently defined in parameter space as the intersection of $(L - 1)$ parallelograms where the parallelogram for each $l \in \{2, 3 \dots L\}$ is the set $(S_l \cap S_1)$. This choice is motivated because the innermost layer has the highest density of hits, resulting in short superpoints. Therefore, S_1 is most likely to restrict patch acceptance and almost always ends up defining two sides of the patch polygon. By anchoring on S_1 , we ensure that the complementary patch is aligned with the seed patch in the vertical direction in the parameter space.

If the seed patch P^i is not rectangular, we invoke the second part of the algorithm. The goal of part II is to choose the superpoints of the complementary patch such that, for each layer, the superpoints of the seed and complementary patches overlap. We build the complementary patch P^{i+1} by left-justifying superpoints S_l^{i+1} to the real line $(z_1^{\min}, z_L^{\min})$, where the superscripts denote the respective endpoints (minima) of S_1^i and S_L^i in the real space. This procedure guarantees $S_1^{i+1} = S_1^i$ and initializes $S_L^{i+1} = S_L^i$. The success of part II is achieved by iterating the $S_{l>1}$ as discussed below.

When part II succeeds, the union of the seed patch and its complementary patch creates a rectangle in the parameter space. The rectangular union is exploited to iteratively create the next seed patch by matching its c_a and c_b to the previous (complementary) patch's c_c and c_d , respectively. These iterations continue until a patch's c_c falls outside the field. This condition corresponds to achieving full acceptance for the S_1 strip. In the $(x = z_1, y = z_L)$ parameter space, we have tiled a columnar region; in the real space, all straight lines passing through S_1 are accepted by the union of the patches comprising the column.

The algorithm continues to raster-scan the parameter space by iteratively building the next columnar tiling adjacent to and to the left of the previous column. This is achieved by right-justifying the next column to the minimum z_1 (left edge) of the previous column. In other words, we create a tiling of patches that raster from maximum to minimum z_L , stepping from maximum to minimum z_1 until the field is covered.

Optimal pairing of seed and complementary patches

We optimize the pairing of the complementary patch with its seed patch to minimize their overlap in parameter space, thereby maximizing the area of the union of the two sets. As the complementary patch is built by left-justifying its superpoints to the desired c_c , we iteratively adjust the z_L value of c_c until the complementary patch has minimum overlap with its seed patch. If this procedure converges, the success of part II is assured.

In this context, the z_L coordinate of each corner, denoted $z_L^{a,b,c,d}$ respectively, is defined as the maximum (for a, b) or minimum (for c, d) value of z_L on the vertical boundaries of the patch, i.e. the boundaries defined by the S_1 strip. Overlap is ensured by requiring $z_L^a(\text{complementary}) - z_L^c(\text{seed}) \equiv \delta^{ac} > 0$ and $z_L^b(\text{complementary}) - z_L^d(\text{seed}) \equiv \delta^{bd} > 0$. As each patch is a convex polygon, ensuring overlap on these vertical boundaries ensures overlap over the entire S_1 strip.

The list of possible z_L^c for the complementary patch is finite and determined by the endpoints of $(L - 1)$ superpoints, which in turn depend on the distribution of hits in the $l > 1$ layers. The quantity $\delta \equiv \min(\delta^{ac}, \delta^{bd})$ varies monotonically with the trial value of z_L^c , therefore, we iteratively optimize z_L^c so that δ is the smallest positive value.

In summary, the complementary patch is moved up or down in parameter space until its overlap with the seed patch is minimized.

Part III - tertiary patch

Part II fails in the rare circumstance that S_1 is a long superpoint (due to a downward fluctuation in the layer 1 hit density) and/or S_l with $1 < l < L$ is a short superpoint (due to an upward fluctuation in the local hit density of layer l). This combination of circumstances forces the seed and/or the complementary patch to have a triangular shape in parameter space, and the geometrical condition required for the definition of the $z_L^{a,b,c,d}$ values in part II is not satisfied.

In this circumstance, the procedure of Sec. "Optimal pairing of seed and complementary patches" does not converge to a rectangular union set of the seed and complementary patches. We invoke part III of the algorithm to create a third "tertiary" patch. It is initialized as a duplicate of the seed patch. The tertiary patch is then right-justified to a trial c_b and the z_1 value of this c_b is iterated until the acceptance gap between the seed and the complementary patches is covered. In other words, S_L of the seed patch replaces S_1 as the anchor for the complementary and tertiary patches, and S_l for $1 \leq l < L$ are iterated for the tertiary patch until the union of the triplet of patches is rectangular.

In terms of the $(x = z_1, y = z_L)$ parameter space, part II adjusts the complementary patch in the vertical direction, while part III adjusts the tertiary patch in the horizontal direction, to eliminate acceptance gaps under almost all circumstances.

Summary of algorithm

We refer to the union of the seed, the complementary, and (occasional) tertiary patches as a superpatch. The tiling procedure based on the rectangular superpatches is highly efficient for containing all particle trajectories. A flowchart summarizing the superpatch algorithm is shown in Fig. 2.

Examples of covers, each built as a set of superpatches to provide 100% acceptance, are shown in Figs. 3 and 4.

Results

We use the simulated data set that was presented in⁷ to demonstrate the performance of the covering algorithm. That data set was generated using a realistic distribution of particle momenta that is based on measurements at the LHC. The number of particles per pp collision is also based on LHC measurements, and the simulation was performed for the beam intensity corresponding to 200 simultaneous collisions per bunch crossing. The processes of particle scattering and decay, as well as ionization energy loss, were included in the simulation⁷. In future work we will extend these results to real collision data.

The distribution of the number of hits in a wedge of pixel sensor data is shown for each layer in Fig. 5. The average number of hits is 70 in the innermost layers, decreasing to 40 in the outermost layer; the maximum fluctuations from wedge to wedge are ± 40 around these averages. The linear density of hits per unit length along the z -axis also fluctuates. The covering algorithm must cope with the complexity and indeterminacy resulting from both types of variability.

In order to build a cover with the minimum number of patches n_P , the algorithm iteratively builds a number of trial patches n_{TP} (of which n_P are retained and the rest deleted). The computational time to build the cover of each wedge is proportional to n_{TP} because each patch is built using a fixed sequence of computations.

The two counts n_{TP} and n_P reflect the complexity associated with the number of hits and their distributions on the different sensor layers of the wedge data. As shown in Fig. 6, the average values n_{TP} and n_P are $\overline{n_{TP}} \sim 70$ and $\overline{n_P} \sim 17$, respectively. The ratio of averages $\overline{n_{TP}}/\overline{n_P} \approx 4$ reflects the iterative optimization of patch overlap discussed in Sec. "Optimal pairing of seed and complementary patches" and Sec. "Part III - tertiary patch".

In the absence of complexity and indeterminacy, the number of patches needed to build the cover may be estimated from the average number of hits in the innermost and outermost layers. With 16 points per superpoint, one typically requires 4.4 superpoints in layer 1 and 2.5 superpoints in layer 5; the product corresponds to 11 patches. Our covering algorithm copes with the variable hit patterns in the three additional layers with only a 50% increase in the number of patches.

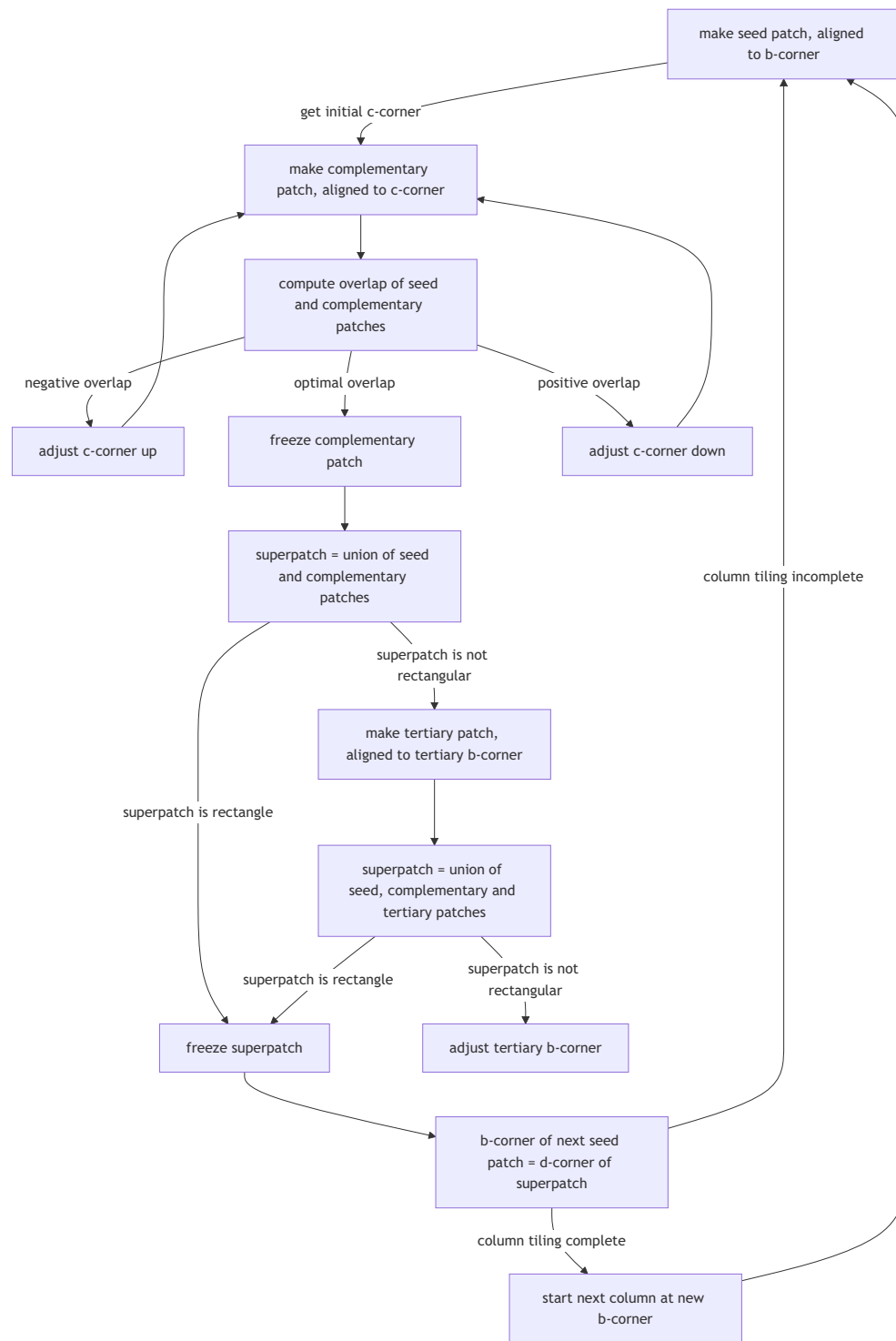


Fig. 2. Flowchart illustrating the superpatch algorithm operating in parameter space.

The most important performance metric is the acceptance of the covers. For each cover, the acceptance ε is defined as the fraction of all straight line trajectories with $-15 < z_0 < 15$ cm and $-50 < z_5 < 50$ cm contained in at least one patch of the cover. It is equivalent to the area of the cover in parameter space (i.e. the area of the union of all patches) as a fraction of the area of the field defined by these bounds. It is useful to show ε versus z_0 , the point of origin of a particle along the beam line, as shown in Fig. 6. The acceptance loss is less than 5 parts per million (ppm) in the $|z_0| < 10$ cm interval, from which 95% of the particles are emitted. The average acceptance loss is 3 ppm over the $|z_0| < 15$ cm interval, assuming a uniform beam-luminosity profile. As a realistic beam-luminosity profile decreases away from $z_0 \sim 0$, Fig. 6 shows that the effective acceptance loss is even smaller. We conclude that the covering algorithm has essentially perfect acceptance for this use case.

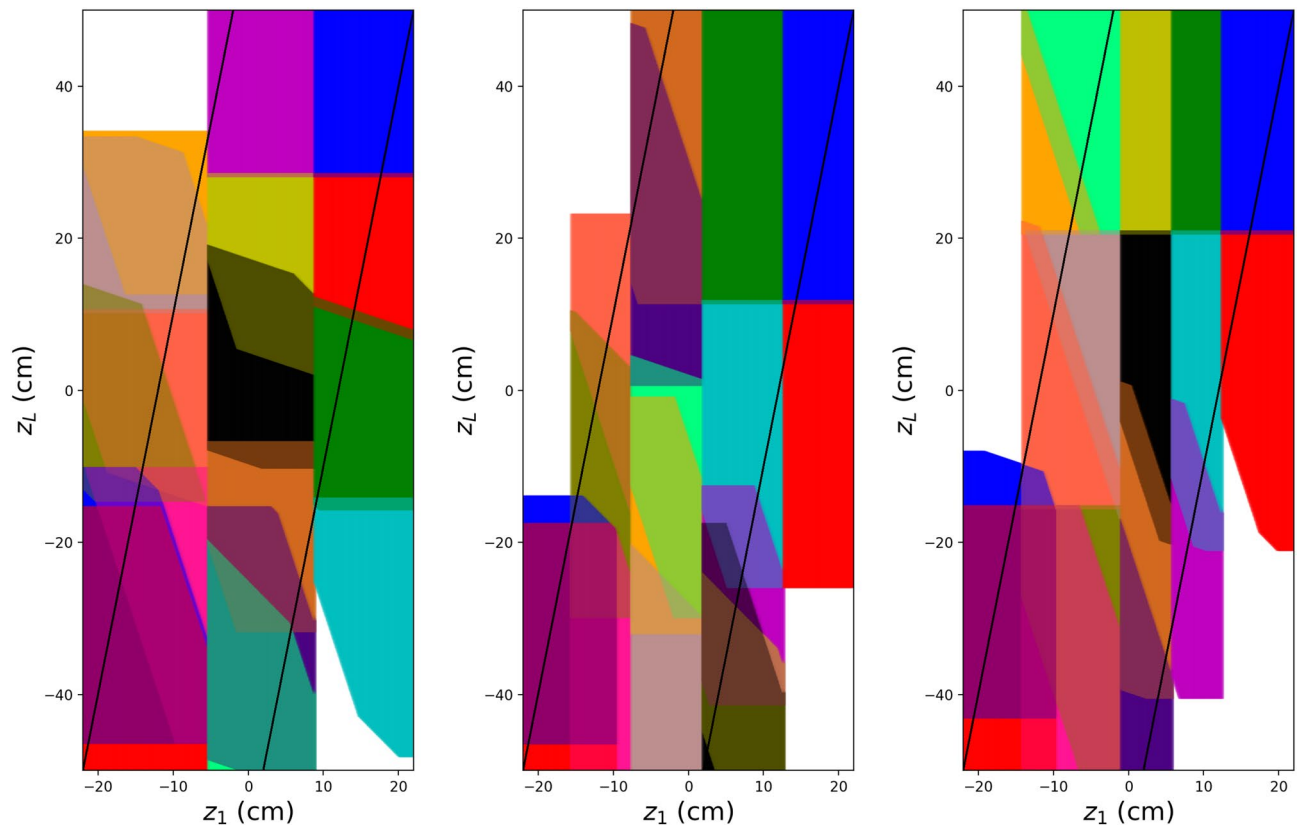


Fig. 3. Examples of covers, each showing the acceptance in parameter space of color-coded patches. Each cover provides 100% acceptance for all tracks in a wedge of pixel hits. The number of patches n_P constituting these covers is 17, the average for the sample (see Fig. 6).

Dependence on feature density

As the number of features in each patch is fixed, the efficacy of the algorithm is represented by the number n_{TP} of the trial patches and the number n_P of the output patches. The smaller these metrics are, the better. These numbers will increase with feature density; the growth rate is an important performance metric.

In the use case of particle tracking at the LHC, the features correspond to hits deposited by the particles in the sensors. The number (and density) of hits grows with the instantaneous luminosity of the colliding proton beams. The latter can be expressed in terms of the average number (μ) of pp collisions per beam crossing; we have presented results for $\mu = 200$.

By varying μ in the simulation, we can proportionately change the number of particles traversing the sensors and the corresponding hit density. Equivalently, we can vary the number of hits per wedge by splitting each event's data into different numbers (n_W) of wedges. We mimic the variation of μ by varying n_W in an inverse proportion.

Another consideration is the curvature of the azimuthal boundaries of the wedge, which are circular concave arcs whose radius is defined by the particle p_T threshold and the magnetic field. We have presented results for $p_T^{\text{threshold}} = 10$ GeV/c and a magnetic field of 2 T. The variation in the algorithm's latency and resource requirement with the p_T threshold is an important consideration for trigger design.

The variation of n_P and n_{TP} with n_W and $p_T^{\text{threshold}}$ is shown in Table 1. For a fixed number of wedges $n_W = 128$, the table shows that the dependence of our supertracking algorithm on the threshold p_T (equivalently, curvature) is weak; for a change in the curvature threshold by a factor of two, the number of patches n_P changes only by 25%. As subsequent processing of each patch is deterministic and requires a fixed computing resource and latency⁸, the slow growth of the number of patches with the curvature threshold represents the total increase in subsequent computing.

Table 1 also shows that the ratio n_{TP}/n_P remains in the range 4 – 4.5 over this variation of the curvature threshold. Since the supertracking algorithm requires a fixed computing resource and latency to create one trial patch, n_{TP} represents the total latency of the supertracking algorithm; this changes by 32% for a factor of two variation in the curvature threshold.

To study the performance of the algorithm as a function of μ , we vary the azimuthal size of the wedge while maintaining its shape, i.e., the azimuthal width of the wedge and the curvature of the wedge boundaries are scaled by the same factor. Increasing or decreasing the size of the wedge by a factor of two is equivalent to doubling or halving μ . Table 1 reveals that the adaptive nature of the algorithm enables its superior performance compared to naive scaling. If there were only two sensor layers, doubling the number of hits in each layer would reduce the

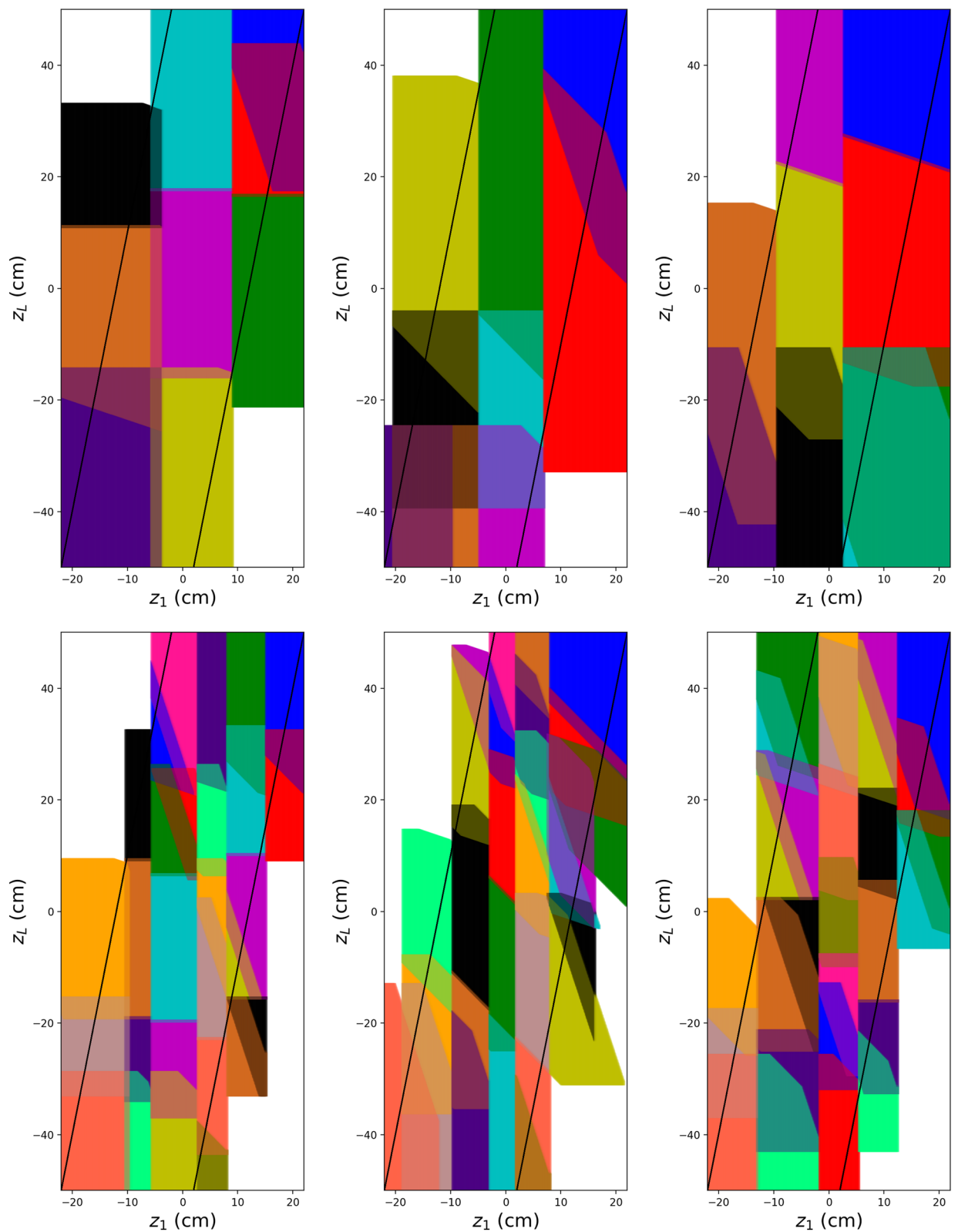


Fig. 4. Examples of covers, each showing the acceptance in parameter space of color-coded patches. Each cover provides 100% acceptance for all tracks in a wedge of pixel hits. The number of patches n_P constituting these covers is 9 (top row) and 28 (bottom row), which are atypical values at the extremes of the n_P distribution (see Fig. 6). The middle plot in the bottom row shows the cover for the wedge data in Fig. 5 (right). A tertiary patch is visible as the magenta patch in the right-most column of this cover.

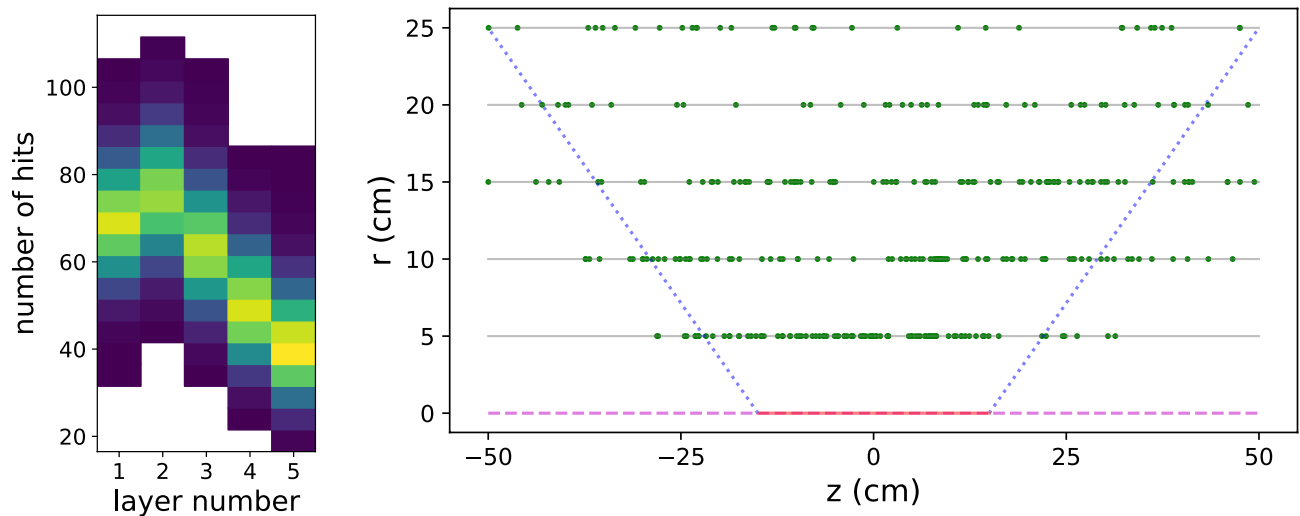


Fig. 5. Illustrations of hit data in wedges. (left) Distribution of the number of hits per wedge in each sensor layer. (right) An example of the spatial distribution of hits (green points) in each layer (horizontal gray lines) of a wedge. The beam axis is shown at $r = 0$ (dashed magenta line). The luminous region of the beam line, $|z_0| < 15$ cm, is indicated by the (solid red) line segment at $r = 0$. The fiducial region for tracking particle trajectories is bounded by the sloping (blue dotted) lines. The cover for this wedge contains $n_P = 28$ patches and required $n_{TP} = 214$ trial patches; this is the highest-complexity wedge in the data set. Its cover is shown in Fig. 4, middle of the bottom row.

acceptance of each patch in the parameter space by a factor of four, thus quadrupling the number of patches n_P in the cover. Each intermediate layer adds constraints on patch acceptance, and one would naively expect n_P to increase by more than a factor of four, given three intermediate layers. In contrast to this expectation of naive scaling, Table 1 shows that the supertracking algorithm achieves the n_P scale factor of three for a factor of two change in μ . The algorithm adapts to local fluctuations in hit density to minimize the number of patches and improve on naive scaling.

The number of trial patches n_{TP} scales by a factor of four when n_W (equivalently μ^{-1}) scales by a factor of two, consistent with naive scaling and reflects the quadratic dependence of the latency of the supertracking algorithm on the number of hits. The fact that n_P scales more slowly than n_{TP} is another indication of the algorithm's adaptive behavior.

Dependence on the number of features in patch

A parameter of choice is the number of features $N \equiv 2^n$ in a patch. A larger N preserves more fine-grained detail in a patch and is suitable when a large number of features and/or a complex pattern of features are required for object identification. The choice of large N transfers the computational complexity from image segmentation to object identification within segments. However, the analysis of fine-grained structure is likely to be inherently more computationally intensive than the image segmentation task, motivating a small value of N .

The disadvantage of a small value of N is that the containment of objects within a segment becomes more difficult, potentially causing loss of acceptance. The optimal configuration depends on the objects to be detected and the signal-to-noise ratio (SNR) in the image. As the SNR decreases, larger values of N are required to obtain good acceptance (ε).

In the LHC use case, we find the loss of acceptance $(1 - \varepsilon)$ to be $(432 \pm 20_{\text{stat}})$ ppm, 3 ppm and 0 ppm for $n = 3, 4, 5$ respectively. Although $n = 3$ still provides high acceptance, the acceptance gain is small for $n = 4 \rightarrow 5$ compared to the gain for $n = 3 \rightarrow 4$. This supports our choice of $n = 4$.

The dependence of the computational latency (indicated by n_{TP}) on N is shown in Table 1. Since each patch is the intersection of five strips in the parameter space of dimension 2, the naive expectation is that the computational latency increases by a factor between k^2 and k^5 when N decreases by the factor k , $N \rightarrow N/k$. Table 1 shows that n_{TP} increases by a factor of ~ 3 for $k = 2$, which is even less than the minimum expectation of 4. Again, this efficacy is because the algorithm adapts to fluctuations in the data.

A similar conclusion can be drawn from the N -dependence of n_P , the number of final patches, which also increases by a factor of 3–5 when N halves.

Note that in the LHC use case, the choice of N has no impact on high- p_T vs. low- p_T track containment; the latter is solely dependent on the p_T threshold that sets the azimuthal boundary of the wedges. In other applications, the dependence of object containment on the choice of granularity will need investigation.

Discussion

The notion of superpatches explains our strategy. Superpatches are unions of patches such that each superpatch has a regular shape in parameter space; in our use case, superpatches are rectangles. The constituent patches of each superpatch are chosen to ensure that the superpatches are disjoint sets. This property enables superpatches to

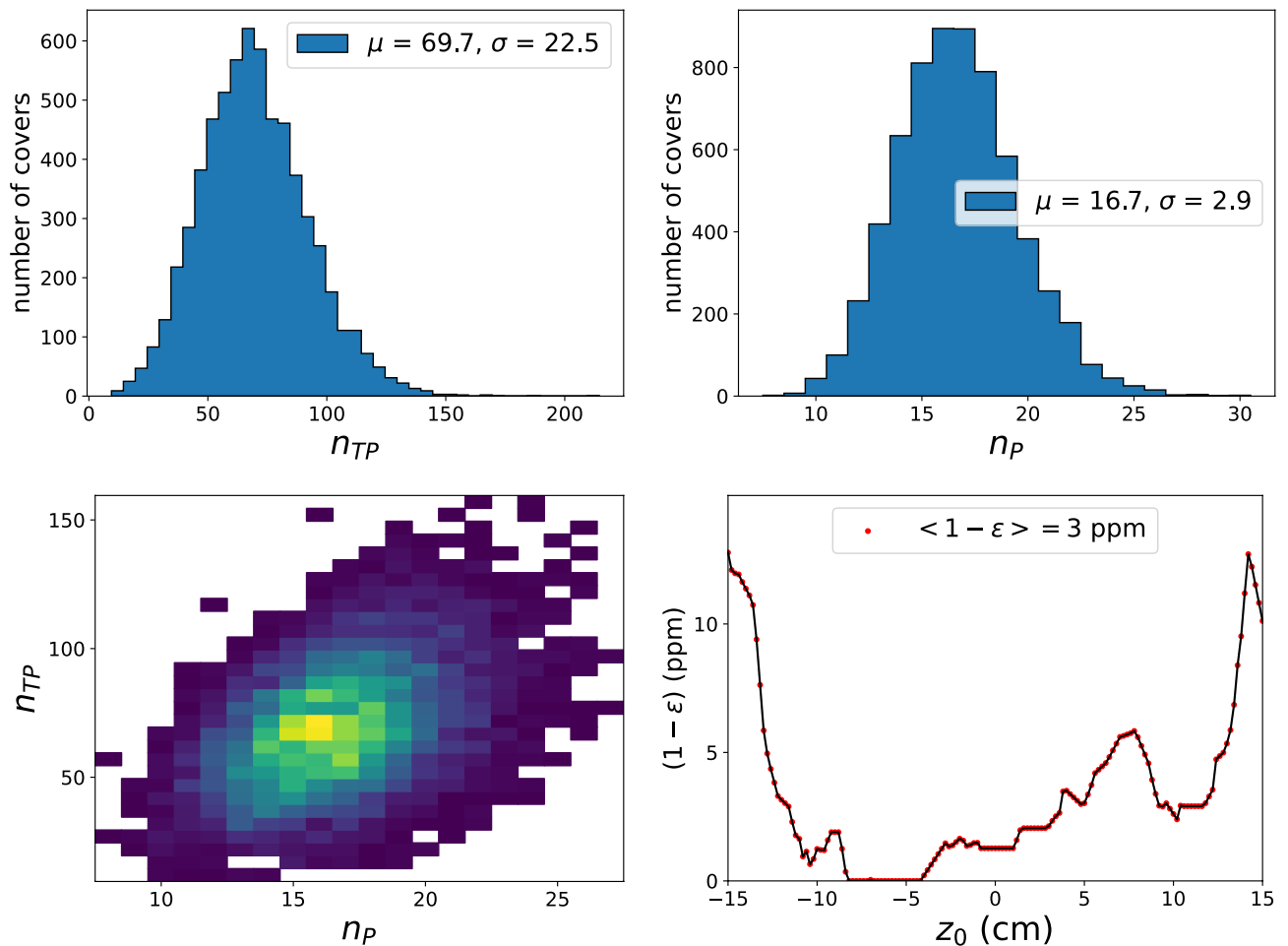


Fig. 6. Performance metrics of the covering algorithm, computing using a simulated data sample of 6400 wedges. (top left) Histogram of the number of trial patches n_{TP} attempted per cover. (top right) Histogram of the number of patches n_P per cover ultimately output by the algorithm. (bottom left) The two-dimensional histogram showing the correlation between n_{TP} and n_P . (bottom right) The lost acceptance, in parts per million (ppm), as a function of z_0 , the point of origin of particles along the beam line. Acceptance (ε) is defined as the fraction of particle trajectories contained in the cover of a wedge. Assuming a uniform underlying distribution of z_0 , the inclusive loss of acceptance is 3 ppm.

perfectly tile the complete parameter space populated by objects, since they provide a cover without overlapping, i.e. the most efficient cover. The details of the strategy describe how to build superpatches out of patches that obey the two stated rules.

Our covering algorithm is highly parallelizable. Each wedge can be processed in parallel. Within each wedge, the tiling of each S_1 column in the parameter space is also parallelizable. As shown in Table 1, the computational complexity (represented by n_{TP}) scales as the square of the size of the feature set. By parallelizing the tiling of the S_1 columns, the latency will scale linearly with the size of the feature set.

The selection of the n_P final patches from the n_{TP} trial patches is an iterative procedure that dominates the computational complexity. We are developing a deterministic procedure, to be presented in a future publication, where the ratio n_{TP}/n_P will be both reduced and constant, significantly improving the computational complexity while building the same superpatches.

In addition to the size of the feature set, computational complexity also depends on fluctuations in the local density of features. Some of this variability will be removed when the iterative step is replaced by a deterministic procedure. Residual effects of local density fluctuations will be investigated with the updated version of the algorithm, which will provide greater insight into computational complexity.

In the context of finding all particle tracks at the LHC, superpatches can be called supertracks because they consist of aligned superpoints and provide acceptance for bundles of proximate tracks. Specifically, the set of supertracks is a partition of the set of all possible tracks.

Supertracking presents a novel and powerful solution to the challenging task of tracking particles. For decades, particle tracking has been performed by directly analyzing the full complexity of the hit collection. This approach has to deal with the maximum computational indeterminacy associated with recognizing large-

scale structures from the small-scale feature map. Computer vision using deep neural networks take the same approach which explains why the networks tend towards billions of training parameters.

Supertracking separates the indeterminacy from the complexity. The construction of supertracks deals with the indeterminacy inherent in the image. However, since supertracks are built from patches and patches are built from superpoints containing $N \times L$ hits, the number of superpoints is much smaller than the number of hits. Effectively, by blocking or coarse-graining over the small-scale structures, the complexity is vastly reduced during the indeterminate phase of our object detection strategy.

Each patch in a supertrack still contains all the small-scale complexity. This is where we reap the benefit of rule (i) creating patches with a fixed number of features. The detection of all tracks in a patch and the computation of their momentum vectors is accomplished by a deterministic unsupervised learning algorithm with no training parameters⁷. The deterministic algorithm is executed with a fixed predictable latency⁸. This is a crucial requirement for real-time applications. A digital circuit implementation with a latency of 250 ns and throughput of 40 MHz at the LHC was demonstrated in⁸. Each patch can be processed independently, enabling a massively parallel architecture for object detection.

Furthermore, it was shown^{7,8} that all high-momentum particles within a patch can be detected with 99.94% efficiency and their momenta evaluated with a spurious rate of 0.3 per million patches. In this paper, we have shown that all high-momentum particles are contained within patches with essentially 100% efficiency. In combination, our strategy detects and measures all particles of interest with negligible exception.

Finally, the challenge at the LHC is to detect thousands of particles in a point cloud of size 10^5 at the 40 MHz rate and with $\mathcal{O}(1 \mu\text{s})$ latency. This is an unsolved problem. Our cover algorithm is parallelizable and capable of solving this problem when coupled with the track-finder^{7,8}.

Conclusions

We have presented a new paradigm for object detection in a noisy collection of features. The feature set is divided into overlapping subsets called patches, such that each object is guaranteed to be contained in at least one patch. Furthermore, we require that all patches contain a fixed number of features. These choices factorize the computational indeterminacy of the feature set due to its small-scale complexity. The indeterminacy is confined to the patch-making step that operates at large scales, hence the complexity of this step is substantially reduced because the patches aggregate a large number of features. The complexity at small scales within each patch is subsequently handled by a deterministic algorithm that operates identically on all patches to identify objects of interest in real time. We have shown the effectiveness of this paradigm in solving the challenging use case of finding particle trajectories in the extremely dense and noisy environment of high-energy proton collisions at the Large Hadron Collider.

Data availability

The data set used and analyzed during the current study is available from the corresponding author on reasonable request.

Received: 19 January 2025; Accepted: 28 August 2025

Published online: 03 October 2025

References

1. Yu, Y. et al. Techniques and Challenges of Image Segmentation: A Review. *Electronics* **12**, 1199. <https://doi.org/10.3390/electronics12051199> (2023).
2. Minaee, S. et al. Image Segmentation Using Deep Learning: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**(7), 3523–3542. <https://doi.org/10.1109/TPAMI.2021.3059968> (2022).
3. Voulodimos, A., Doulamis, N., Doulamis, A. & Protopapadakis, E. Deep Learning for Computer Vision: A Brief Review. *Computational Intelligence and Neuroscience* **2018**, 7068349. <https://doi.org/10.1155/2018/7068349> (2018).
4. Kadanoff, L. P. Scaling laws for Ising models near T_c . *Physics* **2**, 263 (1966).
5. Wilson, K. G. Renormalization group and critical phenomena. I. Renormalization group and the Kadanoff scaling picture. *Phys. Rev. B* **4**, 3174 (1971).
6. Wilson, K. G. Renormalization group and critical phenomena. II. Phase-space cell analysis of critical behavior. *Phys. Rev. B* **4**, 3184 (1971).
7. Kotwal, A. V. Searching for metastable particles using graph computing. *Sci Rep* **11**, 18543. <https://doi.org/10.1038/s41598-021-97848-6> (2021).
8. Kotwal, A. V., Kemeny, H., Yang, Z. & Fan, J. A low-latency graph computer to identify metastable particles at the Large Hadron Collider for real-time analysis of potential dark matter signatures. *Sci Rep* **14**, 10181. <https://doi.org/10.1038/s41598-024-60319-9> (2024).
9. The ATLAS Collaboration. A detailed map of Higgs boson interactions by the ATLAS experiment ten years after the discovery. *Nature* **607**, 52–59. <https://doi.org/10.1038/s41586-022-04893-w> (2022).
10. The CMS Collaboration. A portrait of the Higgs boson by the CMS experiment ten years after the discovery. *Nature* **607**, 60–68. <https://doi.org/10.1038/s41586-022-04892-x> (2022).
11. Kotwal, A. V. A fast method for particle tracking and triggering using small-radius silicon detectors. *Nucl. Inst. Meth. Phys. Res. A* **957**, 163427. <https://doi.org/10.1016/j.nima.2020.163427> (2020).
12. Ryd, A. & Skinnari, L. Tracking triggers for the HL-LHC. *Annu. Rev. Nucl. Particle Sci.* **70**, 171–195. <https://doi.org/10.1146/annurev-nucl-020420-093547> (2020).
13. Bartz, E. et al. FPGA-based real-time charged particle trajectory reconstruction at the Large Hadron Collider. In IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM) 64–71 (2017). <https://doi.org/10.1109/FCCM.2017.27>.
14. Ali, M. L. & Zhang, Z. The YOLO Framework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection. *Computers* **13**, 336. <https://doi.org/10.3390/computers13120336> (2024).

Acknowledgements

The author thanks Muchang Bahng, Abhishek Karna, Nika Kiladze, Michelle Kwok, Tanish Kumar and David Raibaut for their help in software development and for helpful discussions.

Author contributions

A.V.K. generated the research idea and the algorithm, and wrote the manuscript.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to A.V.K.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025